

Vision-Based Line Following with End-of-Line Detection and Autonomous Landing on a Parrot Mambo Minidrone

Jatin Satyam

School for Engineering of Matter, Transport and Energy

Ira A. Fulton Schools of Engineering, Arizona State

University, Tempe, AZ

Email: jsatyam@asu.edu

Abstract—This report describes a vision-guided flight in which a Parrot Mambo minidrone follows a taped line on the floor using only its downward-facing camera, detects the end-of-line marker, and lands there autonomously. Frames are converted from the camera's native YUV format to RGB, thresholded to isolate the line, cleaned with median filtering, and reduced to a centroid by blob analysis. The offset between the centroid and the image center forms the error signal steering the drone along the path, and the end-of-line marker triggers the switch from path following to a landing sequence. The full detect, follow, and land behavior runs end to end on the physical drone.

Index Terms—Parrot Mambo, Simulink, line following, blob analysis, image processing, Stateflow, autonomous landing.

I. INTRODUCTION

This flight explores a vision-only navigation stack: no optical-flow shortcuts for guidance, no external markers, nothing beyond the downward camera and classical image processing wrapped in a closed control loop. The task is deliberately legible: follow the taped line on the floor, recognize where it ends, and land there.

The Parrot Mambo carries the sensor suite the whole series relies on: an ultrasound sensor timing pulses to the floor for altitude, a pressure sensor for altitude changes, an IMU providing linear acceleration and angular rates, and a downward-facing camera whose optical flow contributes horizontal velocity estimates. Four motors in an X configuration are commanded through motor mixing: collective thrust, a differential between the clockwise and counter-clockwise pairs for yaw, and side-to-side speed imbalances for pitch and roll. The airframe is underactuated, with six degrees of freedom and only four inputs, so every lateral motion begins as a deliberate tilt of the thrust vector.

II. SYSTEM ARCHITECTURE

All flight code was developed in Simulink against the Parrot Minidrone Support Package and deployed to the physical airframe, keeping the model structure the support package expects: an estimator consuming the sensor buses, a flight control system, and reference logic that the author modified per task. Structuring the work as small reusable subsystems made iteration and debugging on hardware considerably less painful.

The model is organized into the subsystems the vision loop needs: a camera subsystem receiving YUV frames and converting them to RGB, a line-tracking subsystem running the color threshold, median smoothing, blob analysis, and error computation, and reference logic that turns those errors into position commands for the flight control system.

III. METHOD

A. Vision pipeline

Each frame is thresholded against bounds on the color channels to produce a binary mask of the line. A median filter removes speckle while preserving the line's edges, and blob analysis retains the largest connected region, returning its centroid in pixel coordinates. Comparing the centroid with the image center yields lateral and longitudinal error signals.

B. Steering and landing logic

The binary frame is additionally sampled in fixed probe regions: left and right windows plus several forward probe lines whose occupancy values feed a persistent-state controller. That controller advances the position reference along the line, corrects heading from the left-right imbalance, and carries an explicit descent trigger: when the forward probes empty out and the lateral balance is within bounds, the end of the line has arrived and the logic hands over to a gradual, rate-limited descent.

IV. FLIGHT TESTING AND RESULTS

In the deployed flight the drone acquires the taped line, tracks it along its length, recognizes the end-of-line marker, and sets down at the path's end without operator input. The full loop, from detection through touchdown, completes in a single continuous flight on hardware.



Fig. 1. Line-following flight: the drone tracking the taped line toward the end of the path.

V. CHALLENGES

- Keeping a narrow line in frame against drift and camera vibration.
- Color thresholds are lighting-sensitive; brightness shifts can split or momentarily erase the blob.
- Distinguishing the end marker from the line reliably enough to commit to a landing.

VI. CONCLUSION

The flight validates the classical vision pipeline as a sufficient basis for path following and terminal landing on a minidrone, and it established the exact front end reused in the moving-platform landing work that concludes the series.

APPENDIX: MATLAB FUNCTION CODE

The following MATLAB Function blocks are the author's custom code from the deployed Simulink model, reproduced verbatim.

Listing 1. *lineFollowerControl*: persistent-state path controller with descent trigger

```
function [z_yaw, x_coord, y_coord, z_coord, vLR_ratio] = lineFollowerControl(v_input)
% v_input = [vL, v1, v3, v5, vR, vE, cL, cR]
% Outputs: x_coord, y_coord, z_yaw, z_coord
persistent x y yaw searching direction startTime descending z_val descentTriggered prevYawError lostLineTime prevTime
% --- User Config ---
descentRate = 0.001;
% Initialization
if isempty(x); x = 0; end
if isempty(y); y = 0; end
if isempty(yaw); yaw = 0; end
if isempty(searching); searching = false; end
if isempty(direction); direction = 0; end
if isempty(descending); descending = false; end
if isempty(z_val); z_val = -1.1; end
if isempty(startTime); startTime = tic; end
if isempty(descentTriggered); descentTriggered = false; end
if isempty(prevYawError); prevYawError = 0; end
if isempty(lostLineTime); lostLineTime = tic; end
if isempty(prevTime); prevTime = tic; end
% Initial 1-second delay
if toc(startTime) < 1
    x_coord = 0; y_coord = 0; z_yaw = 0; z_coord = z_val; vLR_ratio = 0;
    return;
end
% Input unpacking
vL = v_input(1);
v1 = v_input(2);
v3 = v_input(3);
v5 = v_input(4);
vR = v_input(5);
vE = v_input(6);
cL = v_input(7);
cR = v_input(8);
v7 = v_input(9);

% Calculate vL-vR ratio (avoid division by zero)
if max(vL, vR) > 0
    vLR_ratio = abs(vL - vR) / max(vL, vR);
else
    vLR_ratio = 0; % Both vL and vR are zero
end

% --- DESCENT MODE (ongoing) ---
if descending
    z_val = z_val + descentRate;
    z_val = min(z_val, 0);
    x_coord = x;
    y_coord = y;
    z_yaw = yaw;
    z_coord = z_val;
    if z_val == 0
        descending = false;
    end
    return;
end
% --- TRACKING MODE (line visible) ---
if v3 >= 0.71 && ~searching
    % Reset flags if line is found again
    descentTriggered = false;
    searching = false;
    lostLineTime = tic;
    distance = 0.000060 + (1-abs(v1 - v5)/2) * (((3*v3 + 4*vE)/7) ^ 7) * 0.00082;
    dx = distance * cos(yaw);
    dy = distance * sin(yaw);
    x = x + dx;
    y = y + dy;
    % Yaw correction (PD with dt)
    yawError = v1 - v5;
    dt = toc(prevTime);
    prevTime = tic;
    yawDerivative = (yawError - prevYawError) / dt;
    yaw = yaw - (yawError * 0.0022 + yawDerivative * 0.001);
    prevYawError = yawError;
end
% --- LOST LINE LOGIC ---
else
    % Wait 0.1 seconds before reacting
    if toc(lostLineTime) < 0.1
```

```

        % Do nothing, hold position
    % Check descent condition first (only if not already searching)
    elseif v3 < 0.68 && vE <= 0.5 && vLR_ratio <= 0.3 && ~descentTriggered && ~searching
        distance = 0.33;
        x = x + distance * cos(yaw);
        y = y + distance * sin(yaw);
        descentTriggered = true;
        descending = true;
        searching = false;
        prevYawError = 0;
        prevTime = tic;

    % Otherwise, search by yawing
    else
        if ~searching
            searching = true;
            prevYawError = 0;
            prevTime = tic;
            if (vL^2) * CL > (vR^2) * CR
                direction = -1;
            else
                direction = 1;
            end
        end
        if v3 < 0.99
            yaw = yaw + direction * 0.001;
        else
            searching = false;
        end
    end
end

% Output
x_coord = x;
y_coord = y;
z_yaw = yaw;
z_coord = z_val;
vLR_ratio = vLR_ratio;
end

```

Listing 2. find_x_centroid_left (region centroid)

```

function cx = find_x_centroid_left(binaryImage)
%#codegen

[rows, cols] = find(binaryImage);

if isempty(cols)
    cx = 0; % No object detected
else
    cx = mean(cols); % Normal centroid
end
end

```

Listing 3. find_x_centroid_right (flipped region centroid)

```

function cx = find_x_centroid_right(binaryImage)
%#codegen

[rows, cols] = find(binaryImage);

if isempty(cols)
    cx = 0; % No object detected
else
    imageWidth = size(binaryImage, 2);
    cx = mean(imageWidth - cols + 1); % Flipped centroid
end
end

```

ACKNOWLEDGMENT

The author completed this work as part of RAS 546, Robotic Systems II, in the MS Robotics and Autonomous Systems program at Arizona State University, and thanks Prof. Sangram Redkar, along with teaching assistants Sai Srinivas Tatwik Meesala and Rajesh S. Aouti, for guidance on the Parrot Mambo platform, flight-arena logistics, and safe flight operations.

REFERENCES

- [1] MathWorks, “Simulink Support Package for Parrot Minidrones,” MathWorks documentation, 2025.
- [2] MathWorks, “Getting Started with Simulink for Parrot Minidrone Hover,” MathWorks examples, 2025.

- [3] B. Douglas, “Drone Simulation and Control” video series, MathWorks / Control Systems in Practice.